

Software Design — Assignment 1

CRUD User — Technical Documentation

Patcas Rares

April 2, 2026

Contents

1	Introduction	3
2	Tech Stack	3
3	Application Architecture	3
3.1	N-Layer Architecture	3
3.2	Class Diagram — Entities	4
4	CRUD User Implementation	5
4.1	User Entity	5
4.2	UserRepository	5
4.3	UserService	5
4.4	UserController — REST Endpoints	6
5	Security	6
6	Testing	7
7	Conclusions	7

1 Introduction

This document describes the implementation of the **CRUD User** functionality within the Instagram clone project, developed for the Software Design course.

The application is a social media platform where users can post content, comment and vote on posts. My role in the team was to implement user management (CRUD User), unit testing and application security configuration.

2 Tech Stack

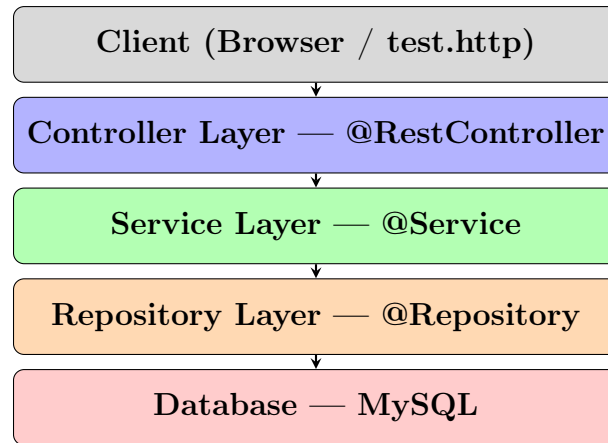
Technology	Version	Role
Java	17	Main programming language
Spring Boot	4.0.3	Main framework — auto configuration
Spring Web (MVC)	4.0.3	REST endpoint exposure
Spring Data JPA	4.0.3	Database access abstraction
Hibernate	7.2.4	JPA implementation — Java to SQL translation
Spring Security	4.0.3	Endpoint security
MySQL	8.0.45	Relational database
Lombok	1.18.30	Auto generation of getters/setters/constructors
Maven	3.x	Build tool and dependency management
Apache Tomcat	11.0.18	Embedded web server
JUnit 5	5.x	Testing framework

Table 1: Tech Stack used in the project

3 Application Architecture

3.1 N-Layer Architecture

The application follows the **N-Layer (Layered Architecture)** pattern, with a clear separation between layers:

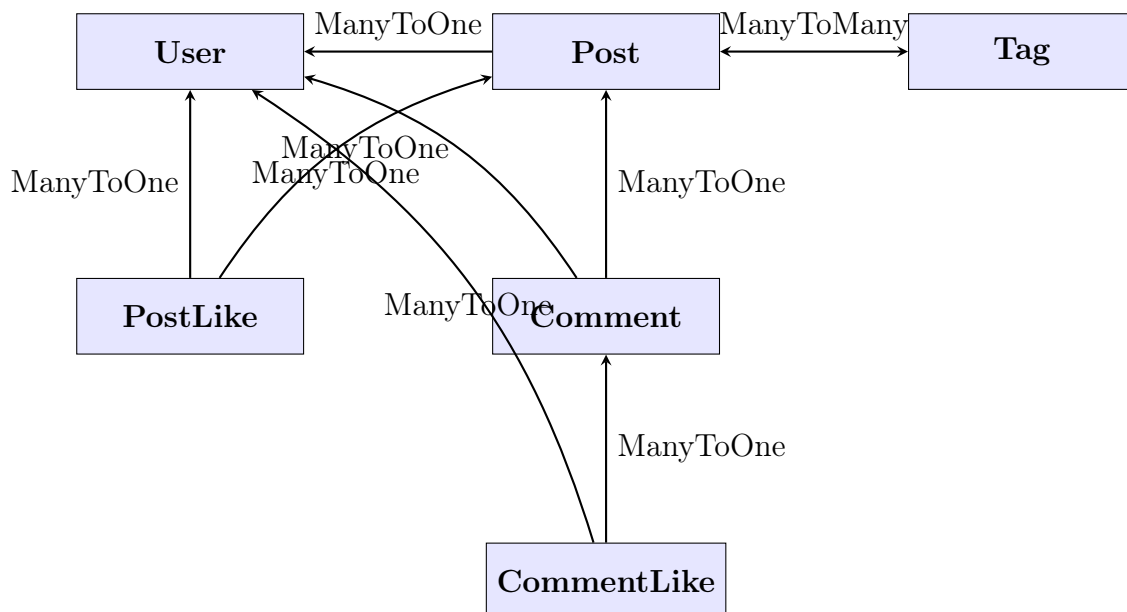


Layer	Annotation	Responsibility
Controller	@RestController	Receives HTTP requests, returns JSON responses
Service	@Service	Business logic, validations, rules
Repository	@Repository	Communication with the database
Entity	@Entity	Mapping Java classes to SQL tables

Table 2: Architecture layers

3.2 Class Diagram — Entities

The project contains the following entities:



4 CRUD User Implementation

4.1 User Entity

The `User` class maps the `users` table in the MySQL database:

Listing 1: User.java

```
1 @Entity
2 @Table(name = "users")
3 @Data
4 @NoArgsConstructor
5 @AllArgsConstructor
6 public class User {
7
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long userId;
11
12    @Column(nullable = false)
13    private String username;
14
15    @Column(nullable = false)
16    private String password;
17
18    @Column(nullable = false)
19    private String email;
20
21    private String role; // "user" or "admin"
22    private Long score;
23    private boolean banned;
24 }
```

4.2 UserRepository

The `UserRepository` interface extends `JpaRepository` and automatically provides CRUD methods:

Listing 2: UserRepository.java

```
1 @Repository
2 public interface UserRepository extends JpaRepository<User, Long> { }
```

4.3 UserService

The `UserService` class contains the business logic for user management:

Listing 3: UserService.java

```
1 @Service
2 public class UserService {
3
4     @Autowired
5     private UserRepository userRepository;
6
7     public User createUser(User user) {
8         return userRepository.save(user);
9     }
10 }
```

```

9      }
10
11     public List<User> getAllUsers() {
12         return userRepository.findAll();
13     }
14
15     public Optional<User> getUserById(Long id) {
16         return userRepository.findById(id);
17     }
18
19     public User updateUser(Long id, User updatedUser) {
20         User existing = userRepository.findById(id)
21             .orElseThrow(() -> new RuntimeException("User not found"));
22         existing.setUsername(updatedUser.getUsername());
23         existing.setEmail(updatedUser.getEmail());
24         existing.setPassword(updatedUser.getPassword());
25         existing.setRole(updatedUser.getRole());
26         existing.setScore(updatedUser.getScore());
27         existing.setBanned(updatedUser.isBanned());
28         return userRepository.save(existing);
29     }
30
31     public void deleteUser(Long id) {
32         userRepository.deleteById(id);
33     }
34 }

```

4.4 UserController — REST Endpoints

HTTP	URL	Method	Description
POST	/users	createUser()	Creates a new user
GET	/users	getAllUsers()	Returns all users
GET	/users/{id}	getUserById()	Returns a user by ID
PUT	/users/{id}	updateUser()	Updates an existing user
DELETE	/users/{id}	deleteUser()	Deletes a user by ID

Table 3: REST Endpoints for User

5 Security

Spring Security was configured through the `SecurityConfig` class, allowing access to all endpoints during the development phase:

Listing 4: SecurityConfig.java

```

1 @Configuration
2 @EnableWebSecurity
3 public class SecurityConfig {
4
5     @Bean
6     public SecurityFilterChain securityFilterChain(HttpSecurity http)
7         throws Exception {
8         http

```

```
9         .csrf(AbstractHttpConfigurer::disable)
10        .authorizeHttpRequests(auth -> auth
11            .anyRequest().permitAll()
12        );
13        return http.build();
14    }
15 }
```

6 Testing

Integration tests were implemented for all CRUD operations using **JUnit 5** and **Spring Boot Test**:

Test	Operation	Result
createUser_ shouldSaveUserInDB	CREATE	Passed
getAllUsers_ shouldReturnList	READ ALL	Passed
getUserById_ shouldReturnUser	READ ONE	Passed
updateUser_ shouldModifyUser	UPDATE	Passed
deleteUser_ shouldRemoveUser	DELETE	Passed
contextLoads	Context	Passed

Table 4: Test results — 6/6 passed

All **6 tests** passed successfully in **823ms**.

7 Conclusions

Within this assignment the following was implemented:

- **Entity Mapping** — 6 entities correctly mapped with JPA relations
- **CRUD User** — all 5 operations implemented and functional
- **Testing** — 6 integration tests, all passed successfully
- **Security** — Spring Security configuration
- **Clean Code** — layered architecture, naming conventions respected